CUDA lecture

CUDA - Compute Unified Device Architecture

Why use GPUs?

- Ideal for tasks that require high parallelism
- Can handle thousands of threads simultaneously
- CPUs in contrast excel at sequential tasks

Terminology

Host - the CPU and its memory

Device - the GPU and its memory

Steps

1. Copy input data from CPU memory to GPU memory
2. Load GPU program and execute, caching data on chip for performance
3. Copy results from GPU memory to CPU memory

`nvcc`

- Is the NVIDIA compiler
- Can be used to compile code with no device code

• Standard C ”hello world” program!

```
int main(void) {
          printf("Hello World!\n");
          return 0;
     }
```

```
$ nvcc 00-hello_world.cu -o 00-hello_world
```

```
__global__ void helloWorld(void) {
    printf("Hello from thread %d from block %d\n",
          threadIdx.x, blockIdx.x);


nt main(void) {
    mykernel<<<10,100>>>();

    return 0;
    }
```

- The `__global__` keyword indicates a function that runs on the device (GPU) and is called from host code (CPU)
- `nvcc` separates code in device code and host code
- Device functions are processed by the NVIDIA compiler
- Host functions are processed by the standard host compiler

Hello World! with Device Code

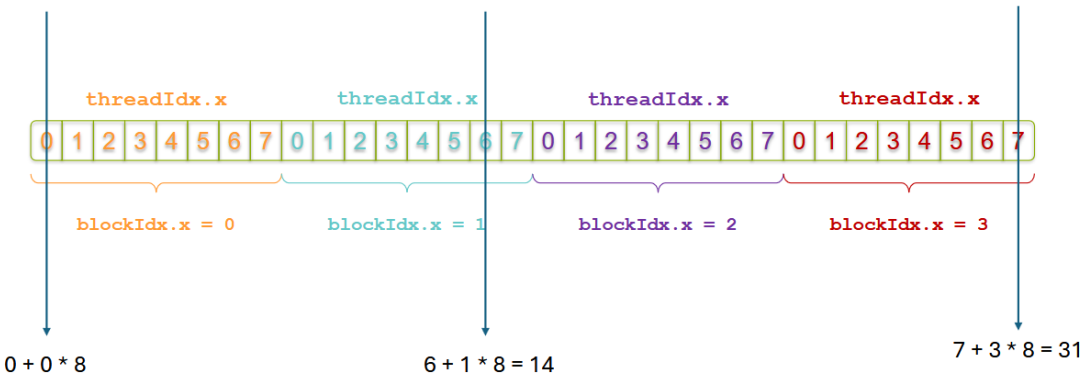
```
helloWorld<<<1,1>>>();
```

- Triple angle brackets mark a call from *host* code to *device* code
  - Also called a "kernel launch"
  - Parameters (1,1) indicate the number of thread block and the number of threads in each thread block
- That's all that is required to execute a function on the GPU!

- First parameter is the number of thread blocks, second parameter is how many threads are in each thread block

- Assume `add<<<4, 8>>>(A, B, C, N)`

```
int index = threadIdx.x + blockIdx.x * blockDim.x;
```

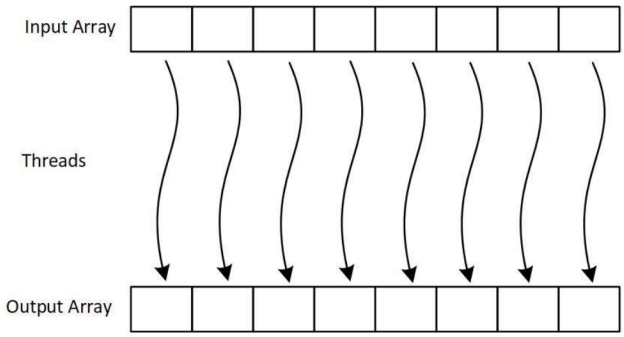


GPU memory management

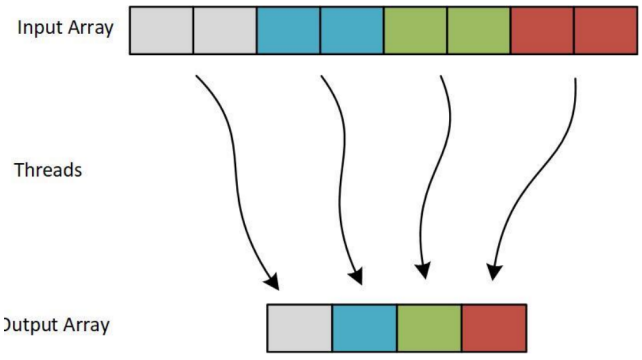
- `cudaMalloc()`
- `cudaMemcpy()`
- `cudaFree()`

Parallel communication patterns

- There are different kinds of parallel communication patterns and they are about how to map tasks (threads) and memory
- **Map**
  - Same operation is independently applied to every element of the dataset
  - No dependencies between elements (embarrassingly parallel)
  - No communication required between the threads
  - Very effective because workload is evenly distributed across threads

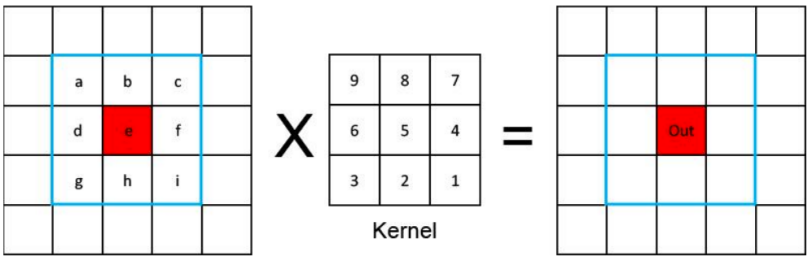


- **Gather**
  - Data is collected from multiple memory locations into a single output dataset
  - Unlike the map pattern, the gather pattern often involves non-contiguous memory accesses
  - Use cases:
    - Reorganizing datasets (e.g., shuffling, grouping, or sorting by specific criteria)
    - Graphics and Rendering: Gathering vertex or texture data for 3D transformations or rendering pipelines
    - Matrix Operations: Extracting rows, columns, or specific elements for sub-matrix computations



- **Reduce**
  - Combine elements into a single result (ex by sum, multiplication)

    - In CUDA, this is implemented by assigning threads to parts of the dataset and then perform a **hierarchical reduction** in shared memory
        - Partial results are iteratively combined until only one result remains
- **Scatter**
    - Distributes data to specific locations in the output dataset
    - Unlike the gather pattern, which collects data, scatter focuses on placing data in non-contiguous or irregular memory locations
    - Use cases
        - Sparse Matrix Representations: Distributing values into specific non-zero locations in sparse matrices
        - Data Partitioning: Splitting datasets into groups or buckets based on a condition or key
- **Stencil**
    - Is the convolution operation from CNNs
    - Is used for calculations where each output depends on the input and its neighbours



- Use cases
    - Image processing
    - Simulation of phenomena which affect each other

Coalesced memory access

- When threads access consecutive memory addresses
- Efficient because it minimizes the number of memory transactions and maximizes memory bandwidth utilization

Shared memory

- Small, fast memory space shared by all threads in a block
- Use cases

- **Reducing Redundant Memory Accesses**:
  - If multiple threads need the same data, they can share it in shared memory instead of each thread fetching it separately from global memory.
- **Minimizing Global Memory Latency**:
  - Global memory access is slow compared to shared memory. Using shared memory avoids repeatedly accessing global memory for frequently used data.
- **Enabling Thread Collaboration**:
  - Threads can share intermediate results via shared memory, making it easier to implement cooperative algorithms.